

AutoWP

Documento explicativo del proyecto para el equipo de desarrollo

Versión 2.0 · Revisión tras evaluación de nivel de detalle para implementación con Claude Code

Nota de revisión — Versión 2.0

Esta versión revisa la v1.0 tras una autoevaluación honesta del equipo: el documento original es correcto y útil como visión de proyecto, pero insuficiente para que un agente de desarrollo (o un programador nuevo) construya AutoWP sin rellenar huecos por su cuenta. A continuación se resume esa evaluación y, en la sección 7, el plan concreto para cerrar la brecha.

Lo que ya funcionaba bien (documento v1.0)

- Objetivo del proyecto y el problema que resuelve.
- Filosofía de diseño (fidelidad visual, automatización primero, modularidad, IA opcional).
- Arquitectura de alto nivel y stack tecnológico previsto.
- Listado de módulos principales y su responsabilidad general.
- Próximos pasos identificados para el equipo.

Lo que faltaba (y por qué es importante)

El documento describía QUÉ debe hacer cada módulo, pero no CÓMO: sin interfaces, clases, flujo interno, estados, casos límite, errores, tests, eventos ni modelos de datos, un agente de desarrollo (o programador) tiene que inventar esas decisiones. Y precisamente ahí es donde aparecen inconsistencias entre módulos, contradicciones no detectadas y comportamiento no revisable.

Dos ejemplos concretos de la ambigüedad detectada:

- **Crawler** — el documento decía "crear un crawler", pero no especificaba el motor exacto (Playwright vs. Puppeteer), número de contextos concurrentes, gestión de cookies, ni cómo resolver lazy loading, infinite scroll, Shadow DOM, iframes cross-origin, Service Workers o rate limiting.
- **WooCommerce Export** — el documento decía "exportar productos a WooCommerce", pero no definía columnas exactas del CSV, tratamiento de productos variables y atributos, SKUs duplicados, imágenes externas, categorías duplicadas ni idiomas.

Nota de valoración

Como documento de visión para explicar el proyecto: 9/10.

Como documento para construir el software directamente: 4/10 — no por estar mal, sino por faltarle nivel de detalle operativo.

Nota sobre el alcance de este documento

Este documento se ha elaborado a partir del diagrama de flujo del proyecto y del guion de documentación técnica acordado (ADR, requisitos, arquitectura, módulos, etc.). Da una visión general

clara y honesta del proyecto, pero — como se explica en la Nota de revisión anterior y se desarrolla en la sección 7 — no sustituye a la especificación técnica detallada por módulo, al registro de decisiones de arquitectura (ADR) ni al documento de reglas de comportamiento para el desarrollo (MASTER_PROMPT.md), que deben producirse antes de iniciar la implementación de cada módulo.

1. Visión general del proyecto

AutoWP es una herramienta que automatiza la migración de un sitio web existente (normalmente una landing, catálogo o tienda estática/HTML) hacia un sitio WordPress + WooCommerce totalmente funcional, replicando el diseño visual original y generando automáticamente el catálogo de productos.

El objetivo no es "clonar visualmente" una web y ya está: AutoWP analiza la estructura del sitio, identifica bloques de contenido (héroes, listados de productos, FAQs, formularios, etc.), extrae los productos que encuentra y construye un tema de WordPress editable desde el propio panel de administración, además de dejar el catálogo importado en WooCommerce listo para vender.

Problema que resuelve

- Migrar manualmente un sitio a WordPress/WooCommerce es lento: maquetar el tema, dar de alta productos uno a uno y configurar el catálogo puede llevar días.
- Muchas pequeñas tiendas o negocios tienen su catálogo únicamente en una web estática, un Excel o un HTML sin backend, sin ninguna forma sencilla de pasarlo a un CMS.
- Los conversores genéricos de "HTML a WordPress" no entienden qué es un producto, qué es un bloque decorativo, ni qué partes deben quedar editables para el cliente final.

Filosofía de diseño

- **Fidelidad visual:** el tema WordPress resultante debe verse igual que el sitio de origen.
- **Automatización primero, intervención humana después:** el proceso corre solo de principio a fin, y el resultado queda editable para ajustes posteriores (precios, textos, imágenes, colores).
- **Modularidad:** cada etapa (crawler, analizador, detector de productos, generador de tema, exportador WooCommerce) es un módulo independiente y sustituible.
- **Uso de IA como apoyo, no como dependencia obligatoria:** el sistema debe poder funcionar con reglas/heurísticas cuando no haya IA disponible, y apoyarse en un modelo (Gemini) para los casos ambiguos.

2. Flujo completo del proceso

El proceso se ejecuta en cuatro grandes pasos, tal como recoge el diagrama de referencia del proyecto ("Automated End-to-End Theme & Catalog Deployment Flow"):

Paso	Script / acción	Entrada	Salida
1	export_static_site.js	URL o proyecto original	Carpeta con index.html, styles.css, scripts.js (sitio estático)
2	generate_wp_theme.js	Sitio estático generado en el paso 1	Tema WordPress completo: style.css, functions.php, plantillas
3	generate_products_csv.js	Datos de productos (Excel	products.csv en formato

		con nombre/URL) o detección automática	de importación de WooCommerce
4	docker-compose.yml	Tema generado + CSV de productos	Entorno WordPress + WooCommerce + MySQL corriendo

Una vez levantado el contenedor de WordPress, el resultado final es un sitio local que muestra el mismo diseño y catálogo que el sitio original, con dos puntos de gestión ya disponibles:

- **Editor de productos (WooCommerce):** permite modificar al instante precio, descripción, imágenes y stock de los productos importados.
- **Configurador de estilos del tema:** permite editar de forma global la paleta de colores, tipografía, cabecera y pie de página del tema generado.

Este diagrama es la referencia visual del pipeline, pero cada paso implica módulos más ricos de lo que muestra la imagen (por ejemplo, el paso 1 involucra un crawler completo con Playwright, cola de rastreo, renderizado JS, etc.), que se describen en el apartado 4 y, con detalle operativo, en las specs por módulo mencionadas en la sección 7.

3. Stack tecnológico previsto

Área	Tecnología prevista	Motivo
Crawler / rendering	Node.js + Playwright	Renderiza JavaScript, lazy-load e infinite scroll
Analizador / clasificación	Node.js/TypeScript + heurísticas + IA	Combina reglas deterministas con IA para casos ambiguos
IA de apoyo	Gemini API (uso opcional)	Clasificación de bloques complejos, no imprescindible
Generación del tema	PHP (theme.json, templates, Gutenberg blocks)	Resultado final es un tema nativo de WordPress
Catálogo	WooCommerce (CSV import)	Estándar de comercio electrónico sobre WordPress
Persistencia interna	SQLite	Snapshots, historial y estado, sin depender de infraestructura externa
Entorno de ejecución final	Docker / docker-compose	WordPress + WooCommerce + MySQL reproducible en local
Interfaz de uso	CLI (Node.js)	Comandos para lanzar cada etapa del pipeline

4. Módulos principales del sistema

Cada uno de estos módulos corresponde a un documento técnico independiente dentro de la especificación completa del proyecto (ver estructura SPEC/ en la sección 7). Aquí se resume su responsabilidad para dar contexto antes de entrar en el detalle de cada uno.

Crawler

Recorre el sitio de origen: lee sitemap.xml y robots.txt, gestiona un pool de navegadores Playwright, renderiza JavaScript, resuelve lazy loading e infinite scroll, respeta rate limits, reintenta peticiones fallidas y prioriza una cola inteligente de URLs. Genera snapshots del DOM para las siguientes etapas.

Analyzer

Toma el DOM capturado por el crawler y construye un modelo interno de la página: detecta bloques (cabecera, hero, listados, FAQ, formularios...), clasifica cada bloque con un sistema de confianza, y usa IA solo cuando la heurística no es concluyente. Debe ser explicable: cada clasificación debe poder justificarse.

Product Detection

Estrategias específicas para encontrar productos dentro de una página: microdatos y datos estructurados (schema.org/JSON-LD), patrones HTML repetitivos, análisis de texto, APIs internas del sitio (JavaScript/fetch), y IA como último recurso cuando ninguna señal estructurada está disponible.

WooCommerce Export

Convierte los productos detectados en un CSV compatible con el importador de WooCommerce: variaciones, atributos, categorías, galería de imágenes y metadatos SEO.

WordPress Builder

Genera el tema de WordPress a partir del sitio estático: theme.json, plantillas de bloque (block templates), integración con Gutenberg, soporte de Customizer, menús y widgets, dejando el contenido editable desde el panel de administración.

Integrations

Detecta si el sitio original usa servicios externos (Stripe, PayPal, Firebase, Supabase, Google Maps, Mailchimp, HubSpot, CRM/ERP) e informa al usuario de qué se detectó, dejando preparados los puntos de configuración para que se activen manualmente después de la migración.

AI Engine

Capa que envuelve el uso de Gemini: caché de resultados para no repetir llamadas, control de costes, reintentos y un modo de fallback sin IA para que el resto del sistema siga funcionando si el servicio no está disponible.

Plugin System

Arquitectura de extensión: interfaces y hooks para que se puedan añadir nuevos detectores, nuevos generadores de bloques o integraciones sin tocar el núcleo del proyecto, distinguiendo entre plugins oficiales y de terceros.

Storage

Persistencia en SQLite del histórico de análisis: snapshots de cada rastreo, versionado del modelo de la página e índices para consultas rápidas.

CLI

Interfaz de línea de comandos con la que el usuario dispara cada etapa del pipeline (crawl, analyze, export, build, deploy) de forma independiente o encadenada.

5. Decisiones de arquitectura clave (resumen ADR)

Estas son las decisiones técnicas ya tomadas que el equipo de desarrollo debe dar por firmes al empezar a programar. El registro completo de decisiones, con su justificación detallada, alternativas descartadas y consecuencias, se mantiene en el documento ADR.md (ver sección 7) y se ampliará a medida que surjan nuevas decisiones.

- El renderizado de páginas se hace con Playwright (no Puppeteer ni scraping estático puro), porque se necesita soportar JavaScript, lazy loading e infinite scroll.
- La IA (Gemini) es un módulo opcional y desacoplado: el pipeline debe poder ejecutarse completo sin clave de API, con calidad reducida en la clasificación de bloques ambiguos.
- El almacenamiento interno de estado (snapshots, historial, versionado) es SQLite, no una base de datos externa, para mantener el proyecto autocontenido y fácil de instalar.
- El resultado final se entrega como un tema de WordPress + WooCommerce corriendo en Docker, no como una web independiente, para que el cliente final gestione su sitio desde el panel de WordPress que ya conoce.
- La detección de productos combina varias estrategias en cascada (datos estructurados → patrones HTML → texto → IA), usando siempre primero la señal más fiable disponible.

Si en el desarrollo surge una contradicción entre estas decisiones y el diagrama de flujo original, prevalece la decisión razonada explícitamente por el equipo, dejando constancia por escrito del motivo del cambio en el ADR.

6. Próximos pasos para el equipo

- Completar el ADR (Architecture Decision Record) formal a partir de todas las conversaciones de definición del proyecto, resolviendo cualquier contradicción detectada y dejándolo como documento vivo versionado en el repositorio.
- Desarrollar la especificación detallada de cada módulo (crawler, analyzer, product detection, WooCommerce export, WordPress builder, integrations, AI engine, plugin system, storage, CLI, configuración, reportes) como documentos independientes, con el nivel de detalle descrito en la sección 7.
- Redactar el MASTER_PROMPT.md con las reglas de comportamiento para el desarrollo asistido por IA (ver sección 7).
- Definir el modelo de datos interno completo (esquema SQLite, entidades, relaciones).
- Especificar los comandos exactos del CLI con sus parámetros y ejemplos de uso.
- Redactar el plan de pruebas (unitarias, integración, E2E, rendimiento) antes de empezar la implementación de cada módulo.
- Mantener toda la documentación en un repositorio navegable (por ejemplo, MkDocs Material) en vez de solo en PDFs sueltos, para que quede siempre sincronizada con el desarrollo real del proyecto.

7. Nivel de detalle necesario y hoja de ruta de documentación técnica

Esta sección traduce la autocrítica de la Nota de revisión en un plan concreto. El objetivo es que, antes de implementar cualquier módulo, exista la spec correspondiente con un nivel de detalle tal que no queden decisiones de diseño por inventar durante la programación.

7.1 Qué debe contener la spec de cada módulo

- Interfaces y tipos de entrada/salida (contratos exactos, no descripciones generales).
- Flujo interno paso a paso.
- Estados posibles y transiciones.
- Casos límite explícitos (qué pasa ante cada situación no estándar).
- Errores: tipos, cuándo se lanzan, cómo se recuperan.
- Tests mínimos requeridos antes de considerar el módulo terminado.
- Eventos emitidos (si aplica), para integración con CLI o con otros módulos.
- Dependencias declaradas hacia otros módulos.

7.2 Estructura de documentación propuesta (carpeta SPEC/)

En vez de un único PDF, la documentación técnica se organiza en documentos independientes, uno por módulo o aspecto transversal. No es necesario redactarlos todos antes de empezar: se prioriza el módulo que se vaya a implementar primero (Crawler), y el resto se completa justo antes de abordar cada módulo.

```
00-Vision.md · 01-Rules.md · 02-Architecture.md · 03-Core.md ·
04-Crawler.md · 05-BrowserPool.md · 06-Queue.md · 07-Discovery.md
· 08-Assets.md · 09-CSS.md · 10-JS.md · 11-Images.md · 12-
Storage.md · 13-DOM.md · 14-Analyzer.md · 15-Blocks.md · 16-
Products.md · 17-WooCommerce.md · 18-ThemeBuilder.md · 19-
WordPressBuilder.md · 20-Gutenberg.md · 21-Forms.md · 22-Menus.md
· 23-SEO.md · 24-Blog.md · 25-Users.md · 26-Integrations.md ·
27-AI.md · 28-Gemini.md · 29-Plugins.md · 30-Docker.md · 31-
CLI.md · 32-Config.md · 33-Logger.md · 34-Errors.md · 35-
Reports.md · 36-Tests.md · 37-E2E.md · 38-Performance.md ·
39-Security.md · 40-CodingStandards.md
```

7.3 ADR — Architecture Decision Record

Documento vivo (ADR.md) que registra cada decisión de arquitectura ya cerrada, con contexto, motivo, alternativas descartadas y consecuencias. Ejemplo de formato:

```
ADR-001 — Motor de renderizado: Playwright Estado: Aceptada Motivo: ejecuta JS,
soporta múltiples contextos, permite screenshots, mejor auto-espera que
Puppeteer. Alternativas descartadas: Puppeteer (solo Chromium), scraping estático
(no ejecuta JS). Consecuencias: necesita un BrowserPool con límite de contextos
concurrentes.
```

El registro completo incluye, como mínimo, las decisiones sobre Playwright, SQLite, IA opcional y desacoplada, entrega final en Docker, y estrategia de detección de productos en cascada — todas ellas ya mencionadas en la sección 5, ahora documentadas con su justificación completa en ADR.md.

7.4 MASTER_PROMPT.md — reglas de comportamiento para el desarrollo

Es el documento con mayor impacto práctico al trabajar con un agente de desarrollo asistido por IA: se carga siempre, junto a la spec del módulo en curso, y fija las reglas que no dependen de ningún módulo en particular.

- Nunca asumir comportamiento no especificado: ante un caso no cubierto por la spec, detenerse y preguntar en vez de inventar una solución.
- Nunca eliminar funcionalidad existente sin que se pida explícitamente.
- Nunca romper la interfaz pública de un módulo sin señalarlo como breaking change.
- Sin dependencias circulares entre módulos.
- Sin código duplicado entre módulos (compartir en core/ o en utilidades comunes).
- Todo módulo nuevo requiere tests antes de considerarse terminado.
- Cada módulo debe ser independiente y sustituible.
- La IA (Gemini) es siempre opcional; todo lo que dependa de IA necesita un camino de fallback basado en heurísticas.
- Seguir la spec del módulo al pie de la letra; no "mejorarla" por iniciativa propia.
- Orden de prioridad ante contradicciones: ADR > MASTER_PROMPT > spec de módulo > diagrama de flujo original.

7.5 Orden de trabajo recomendado

Para no bloquear el arranque del proyecto esperando 60 documentos, se recomienda este orden, escribiendo cada spec justo antes de implementar el módulo correspondiente:

1. MASTER_PROMPT.md y ADR-001 a ADR-005 (ya cerrados, se redactan una sola vez al principio).
2. Spec de Core, BrowserPool y Queue (infraestructura común).
3. Spec de Crawler → implementación y tests.
4. Spec de Analyzer y Blocks → implementación y tests.
5. Spec de Product Detection → implementación y tests.
6. Spec de WooCommerce Export → implementación y tests.
7. Spec de WordPress Builder y Gutenberg → implementación y tests.
8. Resto de módulos (Integrations, AI Engine, Plugin System, CLI, Docker) según prioridad de negocio.

Este documento es una introducción de alto nivel más la hoja de ruta de detalle técnico. Antes de implementar cualquier módulo, el programador (o el agente de desarrollo) debe consultar MASTER_PROMPT.md, ADR.md, y la spec técnica detallada correspondiente al módulo en SPEC/.